

Module 15: Exception Handling

Introduction to C++ Programming — Part 2: Intermediate C++

Module Overview

This module reframes a habit students may already have (checking input 'just in case') as a formal, structured tool. The throw/try/catch payoff lands best when contrasted directly against the silent-corruption alternative (e.g. dividing by zero producing `inf` that quietly poisons later results) - make that contrast the spine of the lesson.

Suggested Duration: 60–75 minutes

Learning Objectives

By the end of this module, students will be able to:

- Explain why silent failure (e.g. division producing `inf`) is more dangerous than a visible error.
- Use `throw` to signal an invalid condition with a specific standard exception type.
- Use `try/catch` to respond to an exception without crashing.
- Chain multiple catch blocks, most-specific first, with a `std::exception` safety net.
- Validate a class's invariant inside its constructor so an invalid object can never be created.

Prerequisites

- Module 11 (inheritance, since standard exception types derive from `std::exception`) and Module 14 (`std::stoi`, which throws on bad input).

Materials & Instructor Prep

- 15-exception-handling/examples/01_throw_and_catch.cpp preloaded, alongside a plain (non-exception) version of the same Ohm's Law bug for direct before/after contrast.

Key Vocabulary

Term	Definition
Exception	An object thrown to signal that something has gone wrong, interrupting normal execution until handled.

Term	Definition
throw	Immediately stops normal execution and begins searching for code prepared to handle the thrown exception.
try / catch	try wraps code that might throw; catch runs only if a matching exception was thrown inside it.
std::invalid_argument / std::out_of_range / std::runtime_error	Standard exception types (from <stdexcept>) naming different broad categories of problem.
std::exception	The base class every standard exception type derives from - a final catch (const std::exception&) acts as a safety net.

Lesson Outline & Instructor Talking Points

1. The silent-failure problem, demonstrated (15 min)

- Show computeOhmsLawCurrent(12.0, 0.0) WITHOUT any exception handling - just plain division. Run it and show the inf (or similarly unhelpful) result silently propagate into a later calculation.
- Ask: 'where did this go wrong?' Let the class notice that the mistake happened far from where the wrong answer eventually surfaced - that gap is exactly the problem exceptions solve.

Instructor note: *This before/after contrast is the module's spine. Make sure the 'before' version genuinely runs and produces a confusing downstream result before introducing any exception-handling syntax at all.*

2. throw and catch (20 min)

- Live-code the fix: throw std::invalid_argument(...) inside computeOhmsLawCurrent, then wrap the call in try/catch. Run it and show the SAME bad input now produces an immediate, clear, catchable message.
- Point out explicitly: code after the try/catch keeps running normally - this is recovery, not a crash. Contrast this directly with what an unhandled exception would do (terminate the program) if time allows a quick demonstration.

3. Multiple catch blocks (15 min)

- Run examples/02_multiple_catch_blocks.cpp using vector::at() to trigger std::out_of_range. Ask why order matters here (most specific first) - tie back to Module 4's if/else-if ordering lesson as a direct parallel.

- Introduce `std::exception` as the base every standard exception derives from - explicit callback to Module 11's inheritance, and Module 12's polymorphism (catching by reference dispatches to the right handler based on the actual thrown type).

4. Where exceptions already exist in code they know (10 min)

- Name `.at()` vs `[]` on a vector explicitly - `[]` is unchecked and dangerous (undefined behavior on bad index), `.at()` throws safely. Recommend `.at()` whenever bounds safety matters more than raw speed.
- Connect back to Module 14: `std::stod/std::stoi` throwing `std::invalid_argument` on bad text - this was already lurking in last module's CSV-parsing code, now it has a name.

5. Validating a constructor (10–15 min)

- Run `examples/03_validating_constructor.cpp`. Land the key insight: once Dosage's constructor throws for a negative value, it becomes IMPOSSIBLE to ever hold an invalid Dosage object anywhere in the program - explicitly connect to Module 8's encapsulation principle as the same idea taken further.

Common Student Misconceptions

Misconception	How to Address It
"Exceptions should replace ordinary if/else for routine, expected branching."	Exceptions are for genuinely exceptional, unexpected situations - routine, everyday decisions still belong in if/else, exactly as in Module 4.
"Catching <code>std::exception</code> alone is just as good as catching specific types."	A single broad catch hides WHICH specific problem occurred, making the error message less useful - prefer specific catches, with <code>std::exception</code> only as a final safety net.
"An empty catch block that does nothing is a safe way to 'handle' an error."	Silently swallowing an exception hides real problems just as badly as the silent-failure scenario this module opened with - at minimum, log or print what happened.

Exercises & Module Project

Exercise: 15-exception-handling/exercises/exercise1.cpp - `safeDivide` and `safeAt`, both exercised with try/catch. Project (three tracks): Generic - Safe Grade Averager; EE - Circuit Safety Checker (invalid resistance + overcurrent); Biomedical - Dosage Safety Checker (invalid dose + unsafe total). All three loop, validate input with exceptions, and keep running after a rejected entry instead of crashing.

Facilitation guidance: The EE and Biomedical tracks both introduce a `SECOND` exception type (`std::runtime_error`, for a safety threshold violation, distinct from `std::invalid_argument` for malformed input) - check that students catch both types distinctly, most-specific first, rather than merging them into one generic catch.

Discussion & Comprehension Check Questions

1. Why is a division that silently produces inf more dangerous than one that throws an exception?

Guidance: The bad value can propagate through many later calculations before producing a visibly wrong result, far from where the actual mistake happened - an exception surfaces the problem immediately, at its source.

2. When would you prefer a simple if/else over throwing an exception?

Guidance: For routine, expected branching that calling code is meant to handle as part of normal operation - exceptions are for the unexpected, not the everyday.

Assessment Ideas

- Given a function signature and a description of what should be invalid input, have students write just the throw statement(s) and choose an appropriate standard exception type, without writing the full function.

Answer Key Notes

See [15-exception-handling/exercises/solutions/exercise1_solution.cpp](#).

Wrap-Up & Bridge to Next Module

Bridge forward: "You've validated one value at a time so far. Next module gives you containers built for holding many values, keyed by name instead of position - map and set."

Instructor Tips

- The before/after silent-failure contrast (step 1) is worth protecting over any other single part of this module if time runs short - it's what makes the rest of the lesson feel motivated rather than arbitrary syntax.